

CentraleSupélec

State-Space Modeling and State-Feedback ControlApplication to a Simulated Rotary Inverted Pendulum

Leelou RUSCH

Dynamic Systems and State-Space Control Course

2APHY

Abstract

This lab report details the state-space modeling, performance verification, and state-feedback control design for a rotary inverted pendulum.

First, foundational concepts of full state feedback and pole placement are verified analytically and computationally on a second-order linear system.

Second, the non-inverted pendulum physics-based model is validated under square-wave excitation to analyze the influence of viscous damping parameters (D_r, D_p).

Next, state-feedback balance control laws are synthesized using both Pole Placement (assigning dominant conjugate poles based on second-order time-domain specifications $\zeta = 0.65, \omega_n = 4$ rad/s) and Linear Quadratic Regulator (LQR) optimal control. Systematic tuning of the LQR weighting matrix Q (q_{11} for arm position responsiveness and q_{44} for pendulum deflection damping) is conducted.

Finally, a quantitative performance comparison between Pole Placement and LQR is presented alongside a critical reflection on hardware implementation constraints.

Reproductibility Guide

This project is documented on github :
`https://github.com/l33lou/simulated-rotary-inverted-pendulum`

The project repository is organized as follows:

```
|--- path to question                          % ex : 1.4.3.2
|   |--- matlab script
|   |--- simulink
|   |--- scopes
|--- logos
|--- report.pdf
```

with as many paths to question as needed to get distinct scopes, meaning running once the matlab and smiluink of one folder is enough to get the results from the report.

1.3 Pole placement control: a refresher from a simulation example

1.3.5 State feedback control design of a simulation second-order system

1.3.5.1 Analytical determination of the feedback gain

1. Consider an LTI system described by the second-order ordinary differential equation:

$$\ddot{y}(t) + \dot{y}(t) - 2y(t) = \dot{u}(t) + u(t) \tag{1}$$

Taking the Laplace transform with zero initial conditions yields the transfer function:

$$(s^2 + s - 2)Y(s) = (s + 1)U(s)$$

Thus

$$G(s) = \frac{Y(s)}{U(s)} = \frac{s+1}{s^2+s-2} \tag{2}$$

2. The open-loop poles are the roots of $s^2 + s - 2 = 0$, giving $s_1 = 1$ and $s_2 = -2$. Because one pole lies in the right-half plane ($s_1 = 1 > 0$), the open-loop system is **unstable**.

3. Decomposing $G(s) = \frac{Y(s)}{Z(s)} \cdot \frac{Z(s)}{U(s)}$ with intermediate variable $Z(s)$ such as :

$$\begin{cases} \dfrac{Z(s)}{U(s)} = \dfrac{1}{s^2+s-2} \\ \dfrac{Y(s)}{Z(s)} = s + 1 \end{cases} \implies \begin{cases} \ddot{z}(t) = -\dot{z}(t) + 2z(t) + u(t) & (3) \\ y(t) = \dot{z}(t) + z(t) & (4) \end{cases}$$

4. Defining state variables $x_1(t) = \dot{z}(t)$ and $x_2(t) = z(t)$ (5), we get :
(3) $\implies \dot{x_1}(t) = -x_1(t) + 2x_2(t) + u(t)$
and (5) $\implies \dot{x_2}(t) = \dot{z}(t) = x_1(t)$

Thus the canonical state-space model is:

$$\begin{bmatrix}\dot{x}_1(t)\\ \dot{x}_2(t)\end{bmatrix} = \begin{bmatrix}-1 & 2\\ 1 & 0\end{bmatrix}\begin{bmatrix}x_1(t)\\ x_2(t)\end{bmatrix} + \begin{bmatrix}1\\ 0\end{bmatrix}u(t), \qquad y(t) = \begin{bmatrix}1 & 1\end{bmatrix}\begin{bmatrix}x_1(t)\\ x_2(t)\end{bmatrix} \tag{3}$$

5. The open-loop characteristic equation is :

$$\det(\lambda I - A) = |\lambda I - A| \tag{4}$$
$$= \begin{vmatrix}\lambda + 1 & -2\\ -1 & \lambda\end{vmatrix} \tag{5}$$
$$= \lambda^2 + \lambda - 2 \tag{6}$$
$$= 0 \tag{7}$$

6. Its roots and poles are the eigenvalues $\lambda_1 = 1$ and $\lambda_2 = -2$. As λ_1 is positive, the system is indeed **unstable**.

7. Let's apply full state feedback control law $u(t) = k_r r(t) - Kx(t)$ with $K = \begin{bmatrix}k_1 & k_2\end{bmatrix}$ into the full state-space model :

$$\begin{cases}\dot{x}(t) = Ax(t) + Bu(t)\\ y(t) = Cx(t)\end{cases} \implies \begin{cases}\dot{x}(t) = (A - BK)x(t) + Bk_r r(t)\\ y(t) = Cx(t)\end{cases}$$

$$\implies \begin{cases}\dot{x}(t) = A_{CL}x(t) + B_{CL}r(t)\\ y(t) = Cx(t)\end{cases}$$

with

$$A_{\text{CL}} = \begin{bmatrix}-1 & 2\\ 1 & 0\end{bmatrix} - \begin{bmatrix}1\\ 0\end{bmatrix}\begin{bmatrix}k_1 & k_2\end{bmatrix} = \begin{bmatrix}-1-k_1 & 2-k_2\\ 1 & 0\end{bmatrix}$$

and

$$B_{CL} = Bk_r = \begin{bmatrix}k_r\\ 0\end{bmatrix}$$

8. The equation of the closed-loop model is $P_{CL}(\lambda) = 0$ with

$$P_{CL}(\lambda) = \det\begin{bmatrix}\lambda + 1 + k_1 & k_2 - 2\\ -1 & \lambda\end{bmatrix} = \lambda^2 + (1 + k_1)\lambda + (k_2 - 2) = 0 \tag{8}$$

9. To assign desired closed-loop poles at $\lambda_1 = -1$ and $\lambda_2 = -2$, the target characteristic polynomial is:

$$P_{obj}(\lambda) = (\lambda + 1)(\lambda + 2) = \lambda^2 + 3\lambda + 2 = 0 \tag{9}$$

Equating coefficients between (8) and (9):

$$1 + k_1 = 3 \implies k_1 = 2, \qquad k_2 - 2 = 2 \implies k_2 = 4 \implies K = \begin{bmatrix}2 & 4\end{bmatrix} \tag{10}$$

1.3.5.2 Determination of the feedback gains with Matlab

In the MATLAB script `1.3.5.3/full_stateback_control.m` we compute :

- the eigenvalues of A like in question 6 with `roots(Den)` and `eig(A)`
- the controlability of the system by checking if `rank(ctrb(A, B))==size(A, 1)`
- the values of the matrix gain K with `K = place(A, B, C, D)` like in question 9
- the eigenvalues of A_{CL} that highlights the stability of the system as all their real parts are negative, which is the goal of using stateback feedback

Then, we implement three representations of the system with Simulink : transfer function integrator hierarchy, manual gain-vector feedback and native `State-Space` block.

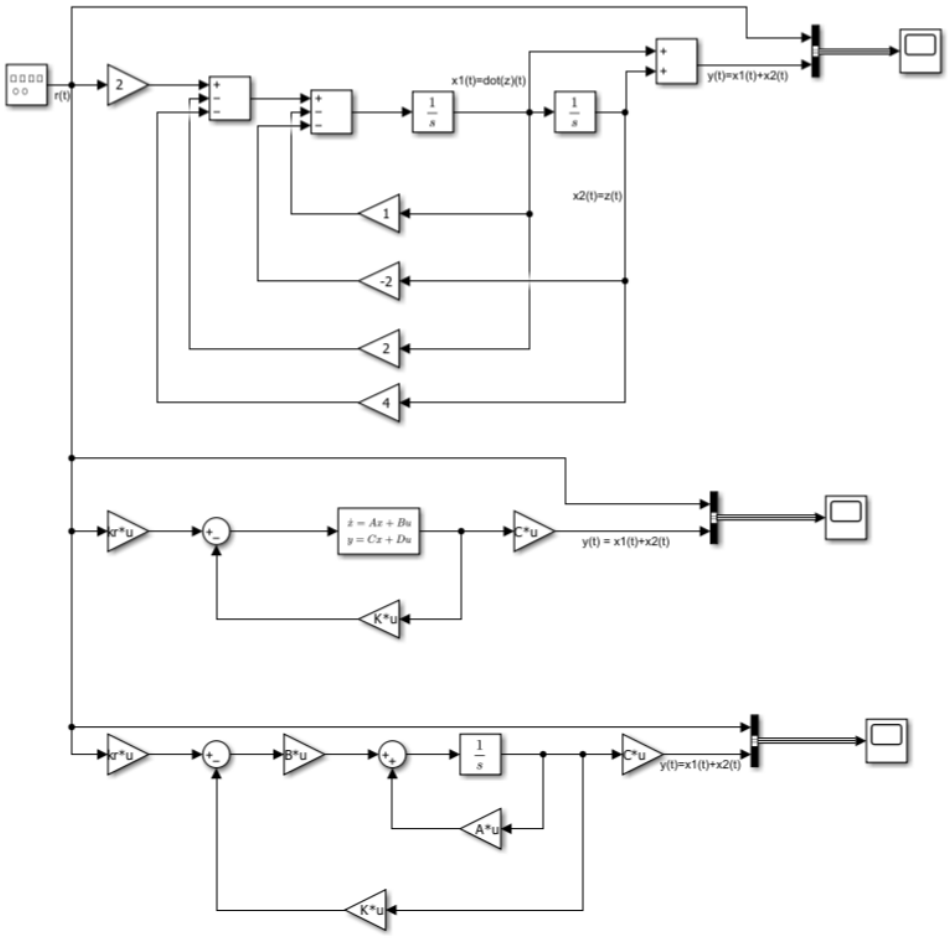


Figure 1: Simulink implementation

These representations are equivalent, as showcased by the following identical scopes :

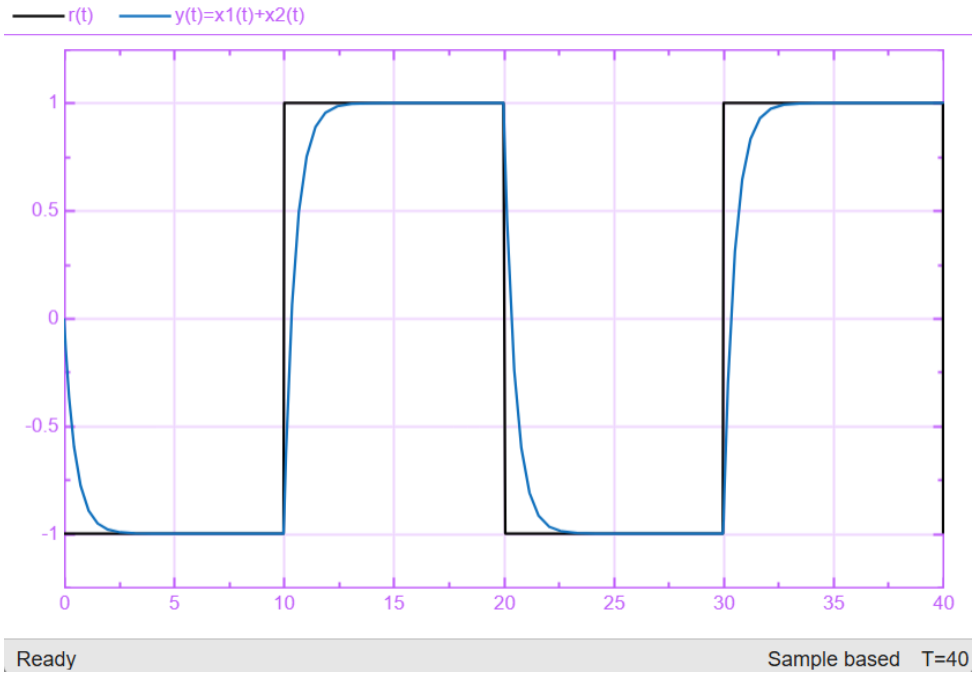


Figure 2: Enter Caption

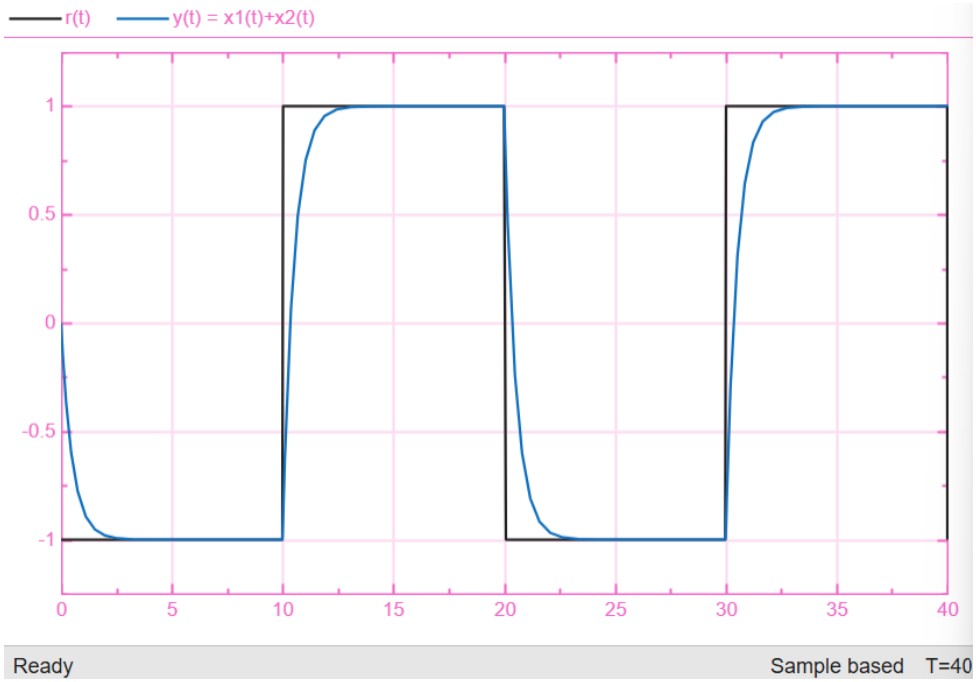


Figure 3: Enter Caption

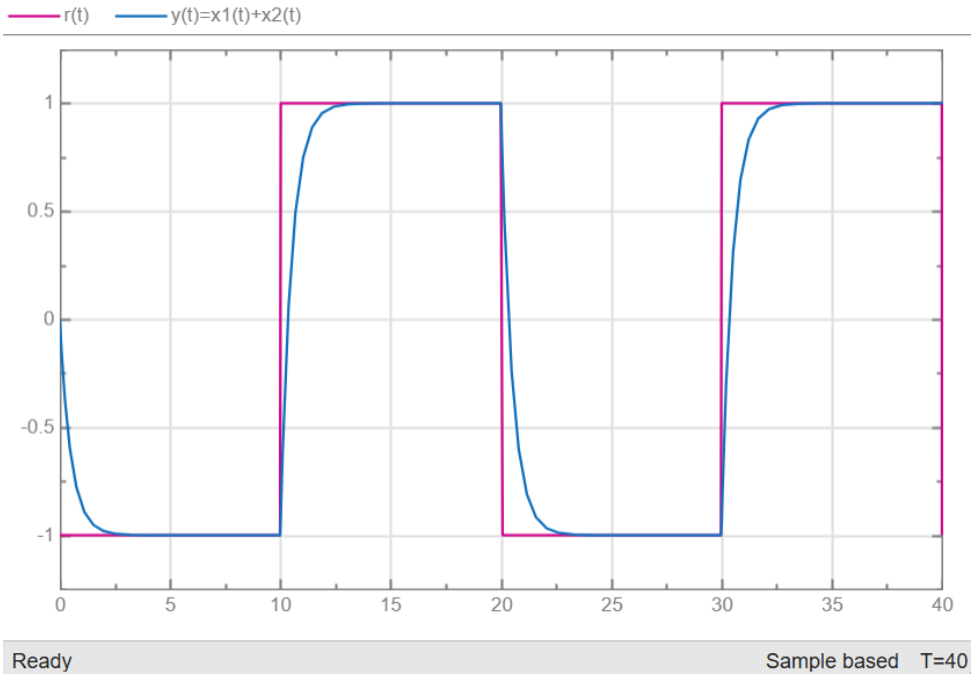


Figure 4: Enter Caption

1.4 Pole placement-based balance control for the inverted pendulum

1.4.1 State-space model of the inverted pendulum

In this section we will study the evolution of the rotary angles θ and α as defined on the following figure :

TODO : ajouter photo schéma

For the non-inverted (pendulum hanging down) configuration, the state vector is chosen as $x(t) = [\theta(t), \alpha(t), \dot{\theta}(t), \dot{\alpha}(t)]^T$.

1.4.1.2 Simulation model validation from step response

The `1.4.1.2.q1-2-3-4-5/setup_non_inverted_pend_ss_model_wo_friction.m` script and the `1.4.1.2.q1-2-3-4-5/simulink2_setup_non_inverted_pend_ss_model_wo_friction.m` Simulink are necessary to answer questions 1 to 5.

1. By computing the Matlab script, we get the following values for A and B :

TODO

2. We input these matrixes as parameters into our simulink :

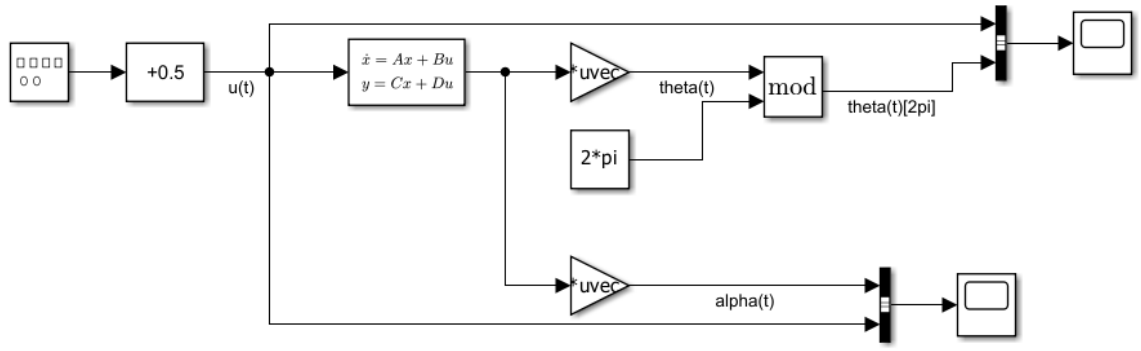


Figure 5: Enter Caption

It was necessary to add the `mod` math function in order to get bounded values for $\theta \in [0; 2\pi]$.

3. 4. 5. By running it, we get the following temporal responses for α and θ :

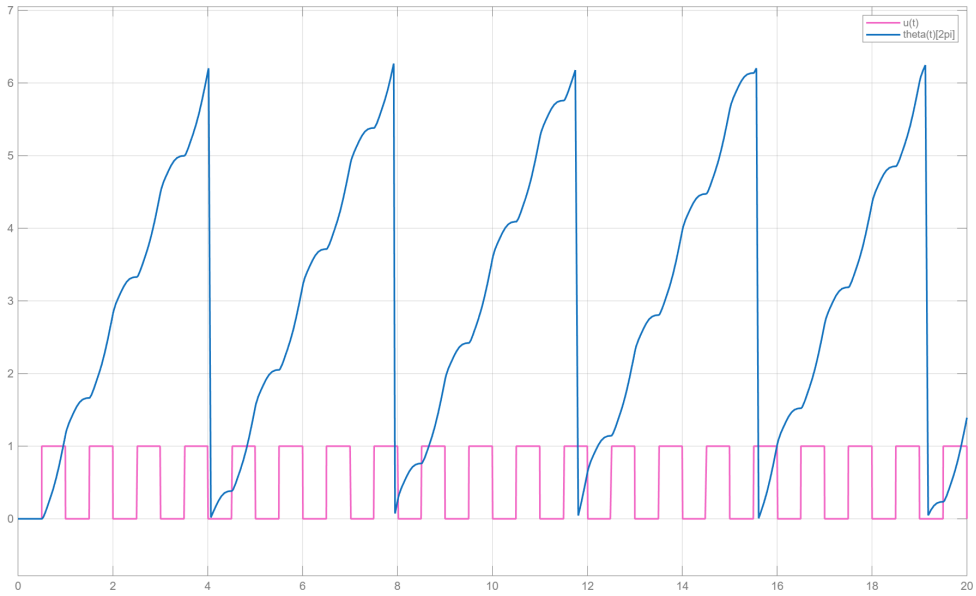


Figure 6: Enter Caption

Thus the rotary arm completes turns around the z_0 axis, in a non-sinusoïdal but periodic way, as it is insluenced by the weight and movement of the pendulum link. The signal possesses two periods : $T_r \approx 4s$ and $T_p \approx 1s$.

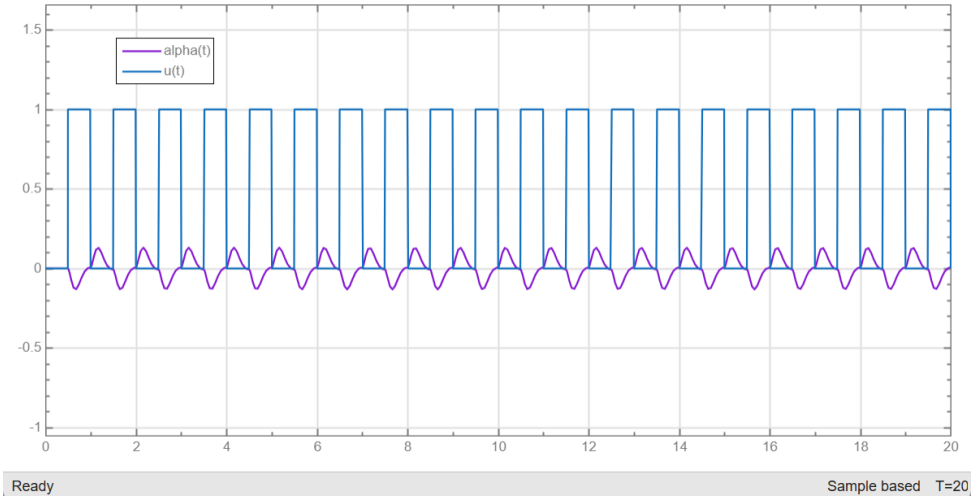


Figure 7: Enter Caption

The pendulum link does not make complete rotations, as α is bounded between ± 0.12 rad. However, it is periodic of period $T_p \approx 1$, which highlights the influence of the pendulum on the rotary arm, and the influence of the rotary arm on the pendulum.

6. 7. In a new script, we set the values of D_r and D_p, the viscous damping coefficients, to 0.5 to observe the effects of the response.
By running `1.4.1.2.q6-7-8/setup_non_inverted_pend_ss_model_wo_friction.m` and imputing the new value of A and B into the simulink :

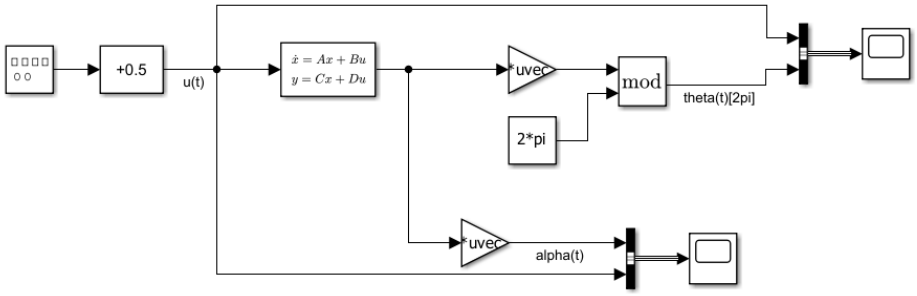
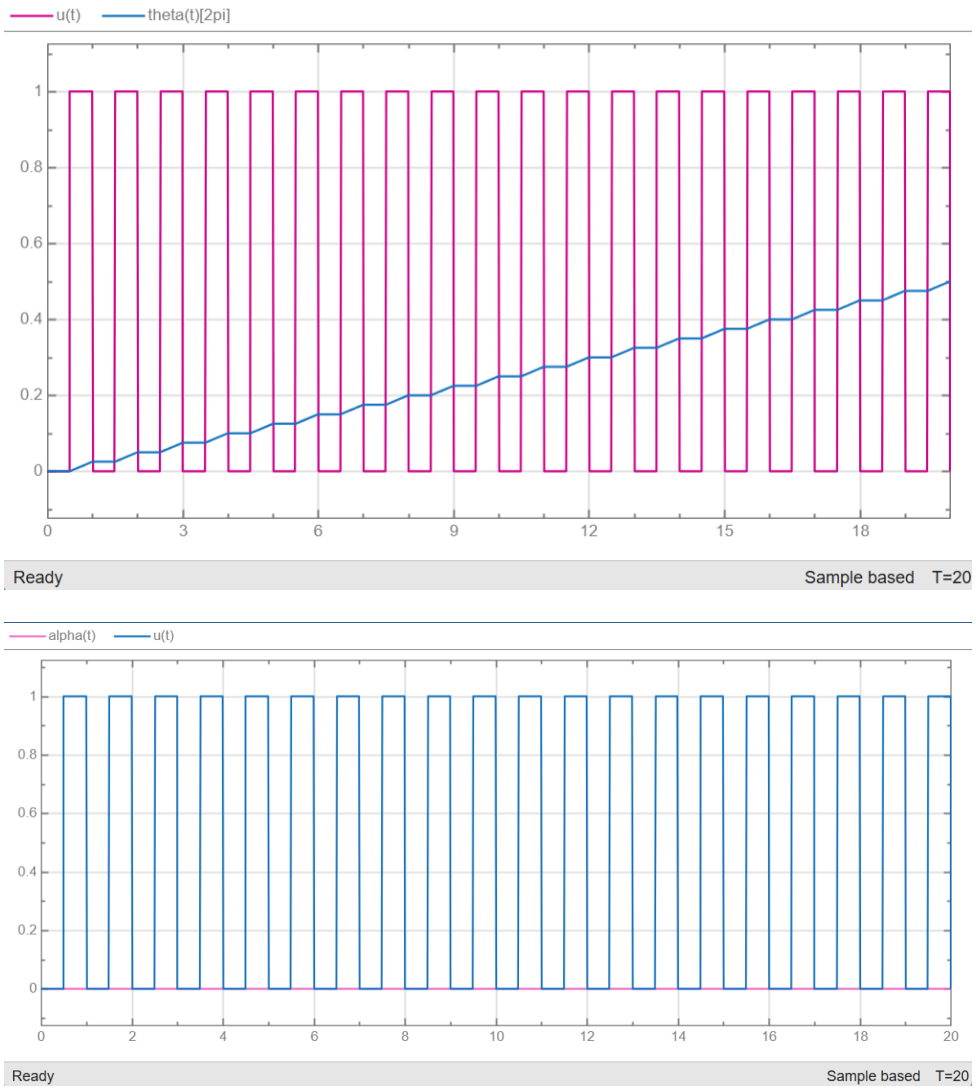


Figure 8: Simulink

we get the following results :



As expected, the rotary arms rotations are restrained and slowed down by friction. Increasing D_r and D_p enhances energy dissipation, decreasing peak overshoot and settling time. Decreasing damping parameters toward zero leads to sustained oscillatory ripples.

8. In our case, unlike a single second-order mode, the non-inverted pendulum displays two distinct coupled oscillatory frequencies : rotary arm motor dynamics coupled with un-driven pendulum swing.

1.4.2 Pole placement control design for the inverted pendulum

1. The inverted pendulum state-space model loaded via `1.4.2-3.q1-4/setup_inverted_pend_ss_model_wo_input.m` returns:

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 149.3 & -14.93 & -4.915 \\ 0 & 261.6 & -14.76 & -8.614 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ 0 \\ 49.73 \\ 49.15 \end{bmatrix}$$

2. Open-loop poles computed via eig(A) are:

$$\lambda_{\text{OL}} \in \{0, -28.64, +10.67, -5.57\} \tag{11}$$

The positive real eigenvalue +10.67 confirms that the open-loop inverted pendulum is **unstable**.

3. The controllability matrix $\mathcal{C} = \begin{bmatrix} B & AB & A^2B & A^3B \end{bmatrix}$ is computed via `ctrb(A, B)`, yielding rank($\mathcal{C}$) = 4. Since the matrix has full rank ($n = 4$), the inverted pendulum is **fully state-controllable**.

4. The performance specifications ($\zeta = 0.65, \omega_n = 4$ rad/s) define the dominant complex-conjugate poles (p_1, p_2):

$$\sigma = \zeta\omega_n = 0.65 \times 4 = 2.60, \qquad \omega_d = \omega_n\sqrt{1-\zeta^2} = 4\sqrt{1-0.65^2} \approx 3.04 \text{ rad/s}$$

Thus,

$$p_{1,2} = -2.60 \pm j3.04 \tag{12}$$

5. To prevent secondary modes from altering the dominant second-order response, non-dominant poles p_3, p_4 are placed far into the left-half plane:

$$p_3 = -40, \qquad p_4 = -45 \tag{13}$$

The characteristic desiered equation is then :

$$(p - p_1)(p - p_2)(p - p_3)(p - p_4) = 0$$

6. Executing `K = place(A, B, P)` with $P = [-2.60{+}3.0396j, -2.60{-}3.0396j, -40, -45]$ yields state-feedback gain vector:

$$K = \begin{bmatrix} -5.08 & 64.22 & -2.36 & 3.74 \end{bmatrix} \tag{14}$$

1.4.3 Simulation of the pole placement balance control

1. Inputing the matrixes resulting from the execution of the code explained in the previous part allow us to simulate the behaviour of our pendulum in `1.4.2-3.q1-4/simulink4_inverted_pend_ss_model_wo_input.m`.

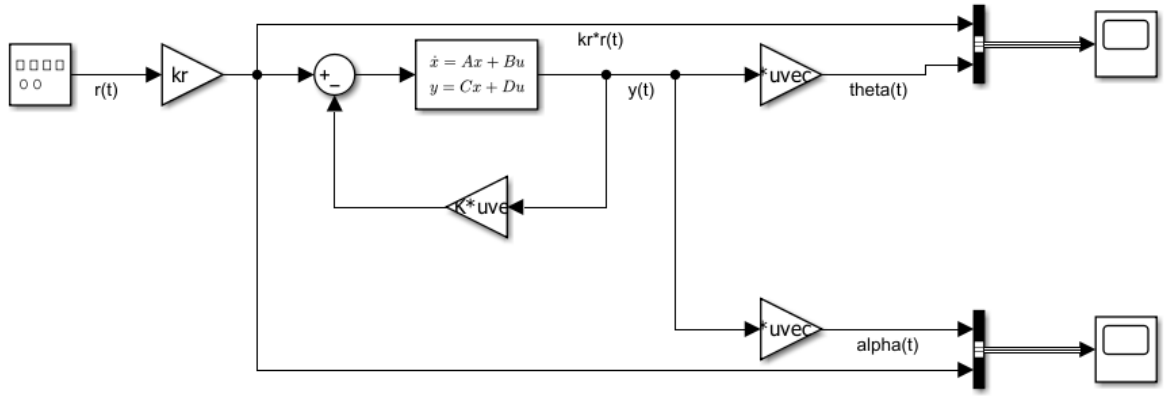
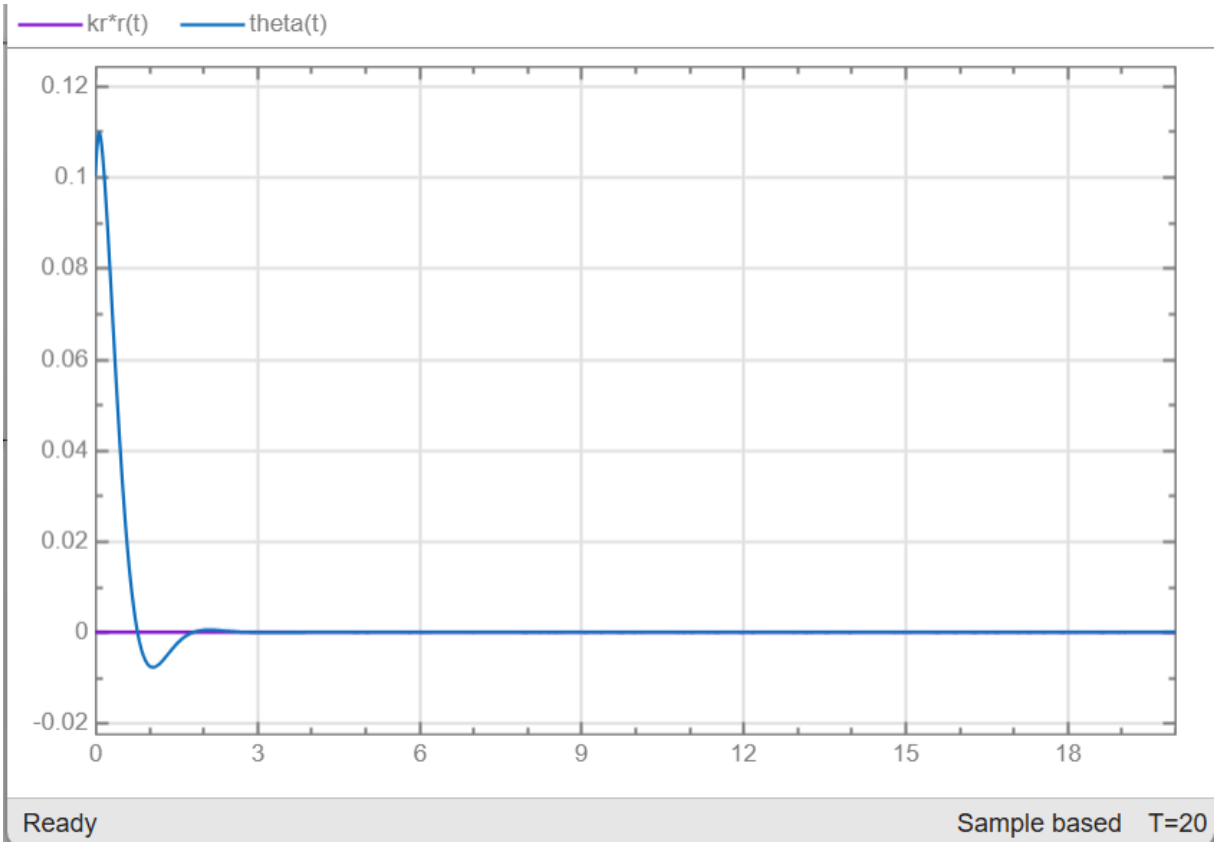


Figure 9: Simulink

We implement the full state-feedback control law by adding $u(t) = -Kx(t) + k_r r(t)$ as the entry signal of our State-Space block.

2. We set the generator block to $r(t)$ a unitary square signal of frequency $f = 0.125$ Hz.

3. First, to study the behaviour of our system on its own, we set $k_r = 0$ so $r(t) = 0$, and the initial conditions of our State-Space block to $x_0 = [0.1; 0; 0; 0]$.



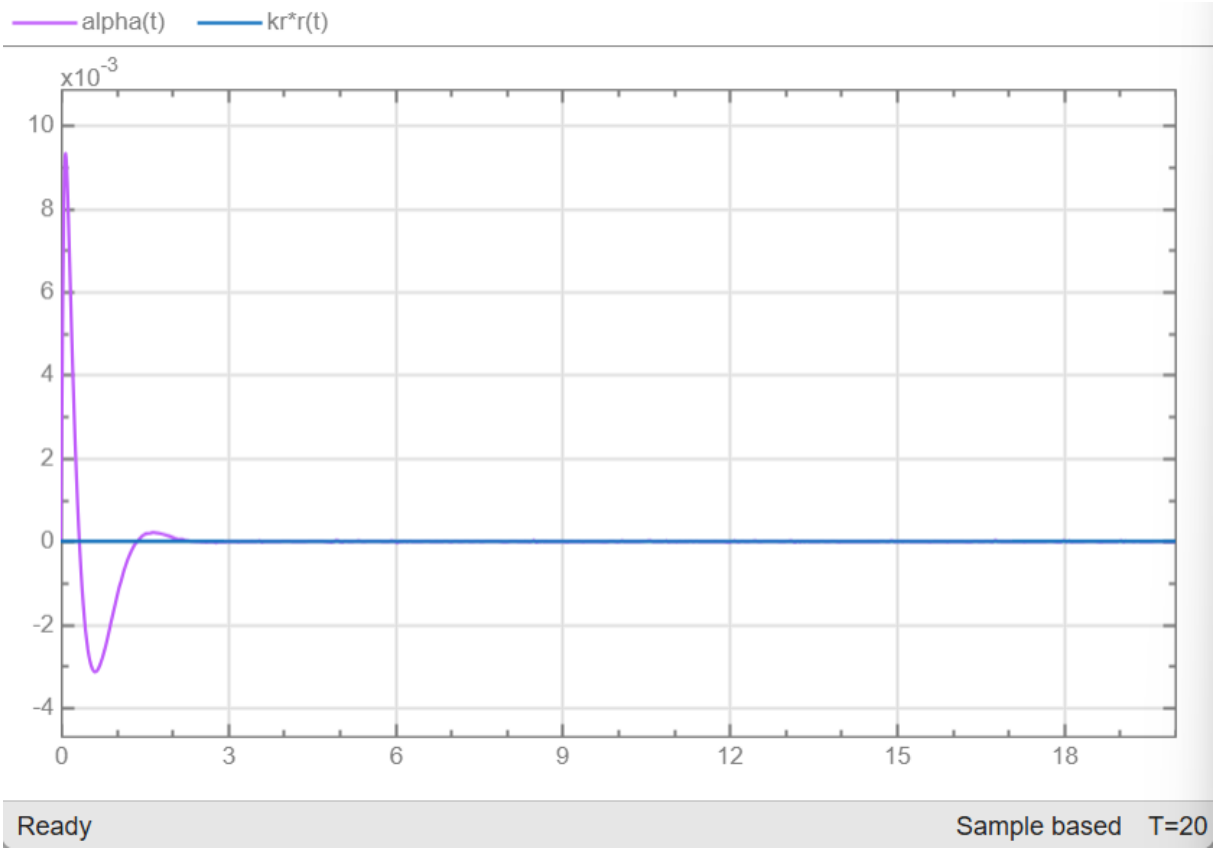


Figure 10: Enter Caption

4. As expected, the angles of the pendulum reach the expected zero (as there is no input / command) thanks to the state-feedback loop. The pendulum remains balanced at zero once it reached this value for both θ and α. There is overshooting in both responses, which is a characteristic of a second-order system.

5. To implement a command $\theta_c = \pm 30^0$ we set the amplitude of our generated signal $r(t)$ to 0.52 rad in
`1.4.3.q5-8/simulink4_inverted_pend_ss_model`.

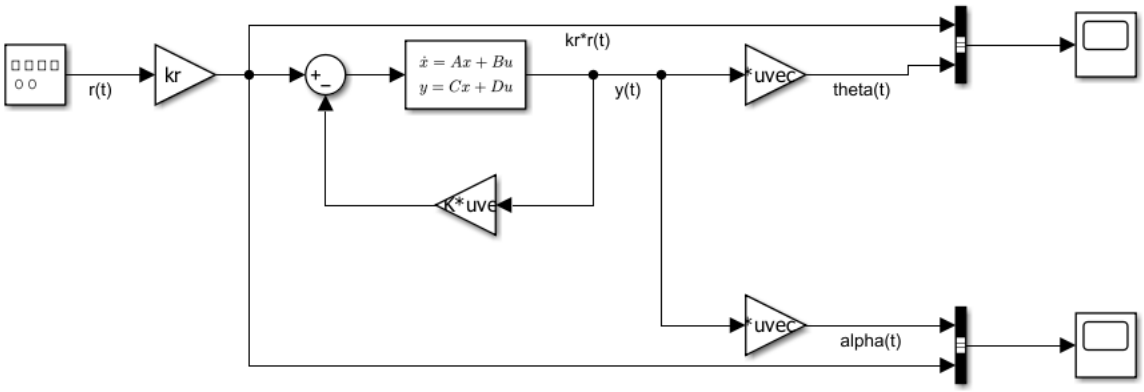
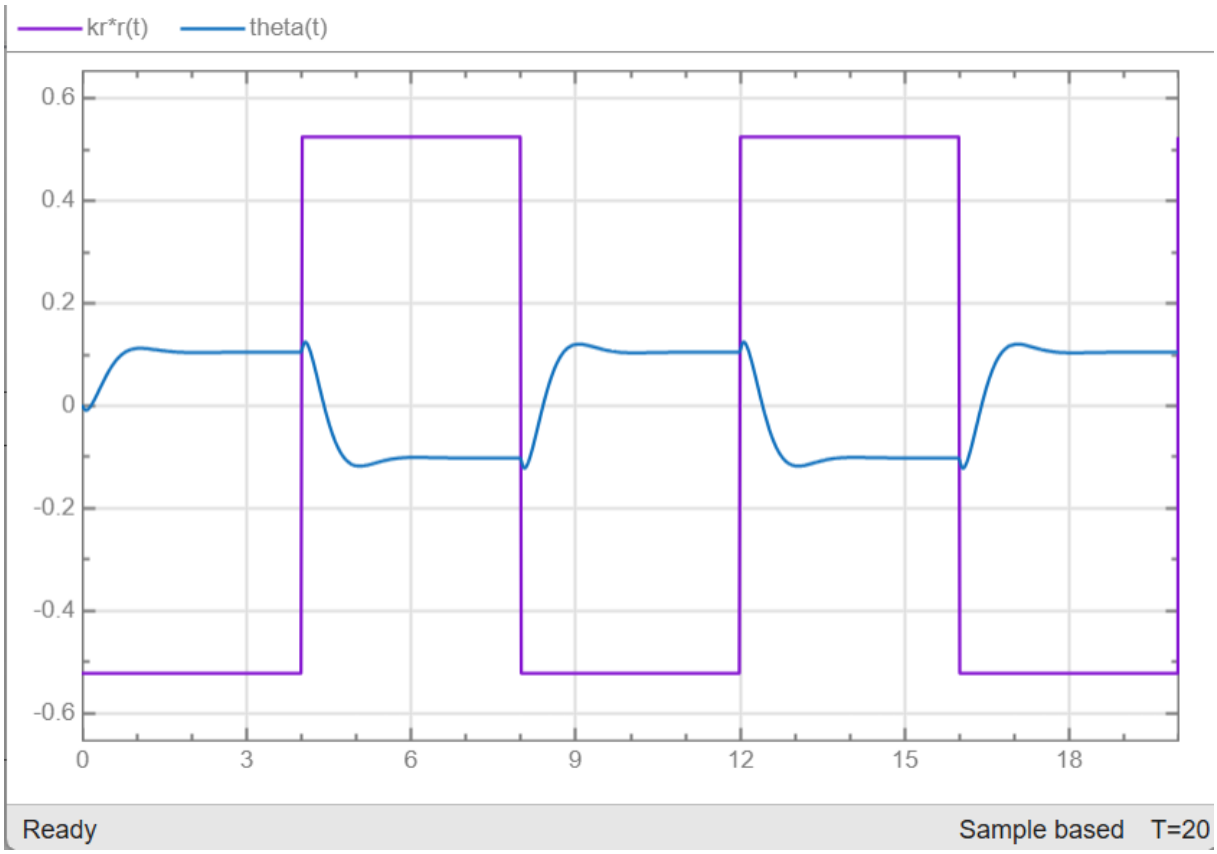
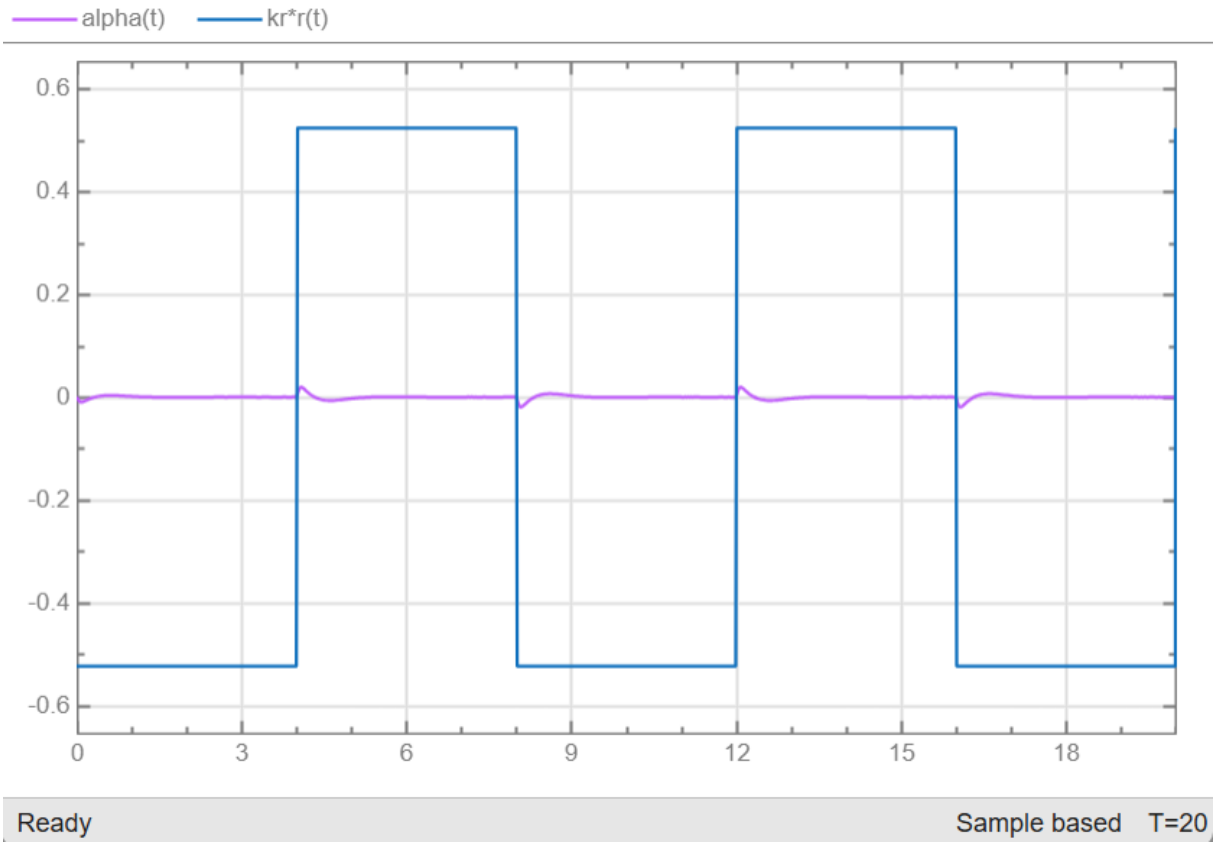


Figure 11: Simulink

6. We get the following scopes :



The expected values of θ aren't reached, but the system does reach a stable value after a settling time of ≈ 1.5s. There is a 20% overshooting.



The pendulum link rests at angle $\alpha = 0$ rad with few variations.

7. The settling time and percent overshoot are $T_s^{5\%} \approx 1.5$s and $D_1 = 20\%$. The settling time from Table 1.1 is correct, but less overshooting is expected. In table 1.1, $D_1 = 6.8\%$.

8. See part 1.5.3.2 question 13.

1.5 Optimal LQR-Based balance control for the inverted pendulum

1.5.3 Design and simulation of the LQR control

With control weight fixed to $R = 1$, the state penalty matrix Q was systematically adjusted in MATLAB using `K = lqr(A, B, Q, R)`, and its effects will be discussed.

1.5.3.1 LQR control design

1. By running `1.5.3.1-2.q1-5/setup_qube2_rotpen.m` script, we get :

$$A = \begin{bmatrix} 0 & 0 & 1.0000 & 0 \\ 0 & 0 & 0 & 1.0000 \\ 0 & 149.2751 & -0.0104 & 0 \\ 0 & 261.6091 & -0.0103 & 0 \end{bmatrix}$$

and

$$B = \begin{bmatrix} 0 \\ 0 \\ 49.7273 \\ 49.1493 \end{bmatrix}$$

2. By computing `eig(A)` we get `[p1 p2 p3 p4 ] = [0 -16.18 16.18 -0.01]`. As `p3` ir real positive, the system is unstable. This is the reason why implementing LQR control is needed.

3. The system is controlable as `rank(ctrb(A, B) == size(A, 1)` is True, which makes the implementation of the LQR control possible.

4. For (Q = diag([1 1 1 1])) and (R = 1), we get (K = lqr(A, B, Q, R) = [-1.00 34.24 -1.23 3.08] by computing `1.5.3.1-2.q1-5/setup_qube2_rotpen.m`.

5. In the same script, `eig(A-B*K)` returns `[-72.75 -1.01 -8.28+3.07j -8.28-3.07j]`. All real parts of the poles are negative, thus the system in state-feedback control with K computed with the LQR method is stable.

6. In the same script, we compute `Kbis = lqr(A, B, Qbis, R)` with `Qbis = diag([5 1 1 1])`. We get `Kbis = [-2.24 37.62 -1.50 3.38]` as expected.

1.5.3.2 Simulation of the LQR-based balance control

1. We use the same script as in the last part, `1.5.3.1-2.q1-5/setup_qube2_rotpen.m`.

2. We implement the Simulink model in `1.5.3.1-2.q1-5/simulink6` with the control law $u(t) = K(k_r r(t) - x(t))$, and K the result of the `lqr` Matlab function.

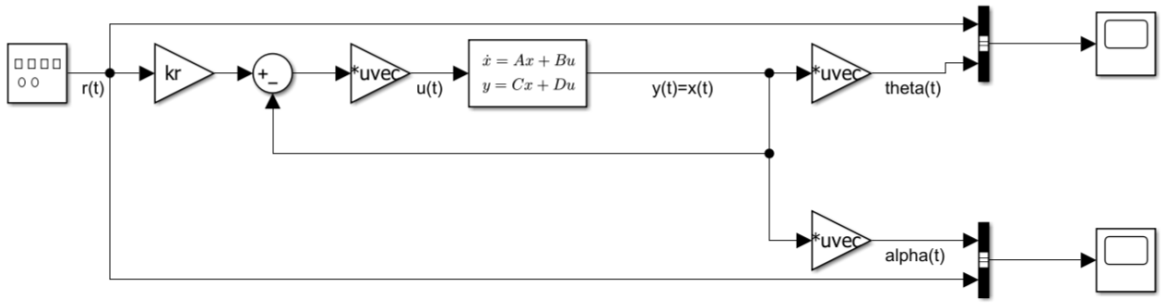
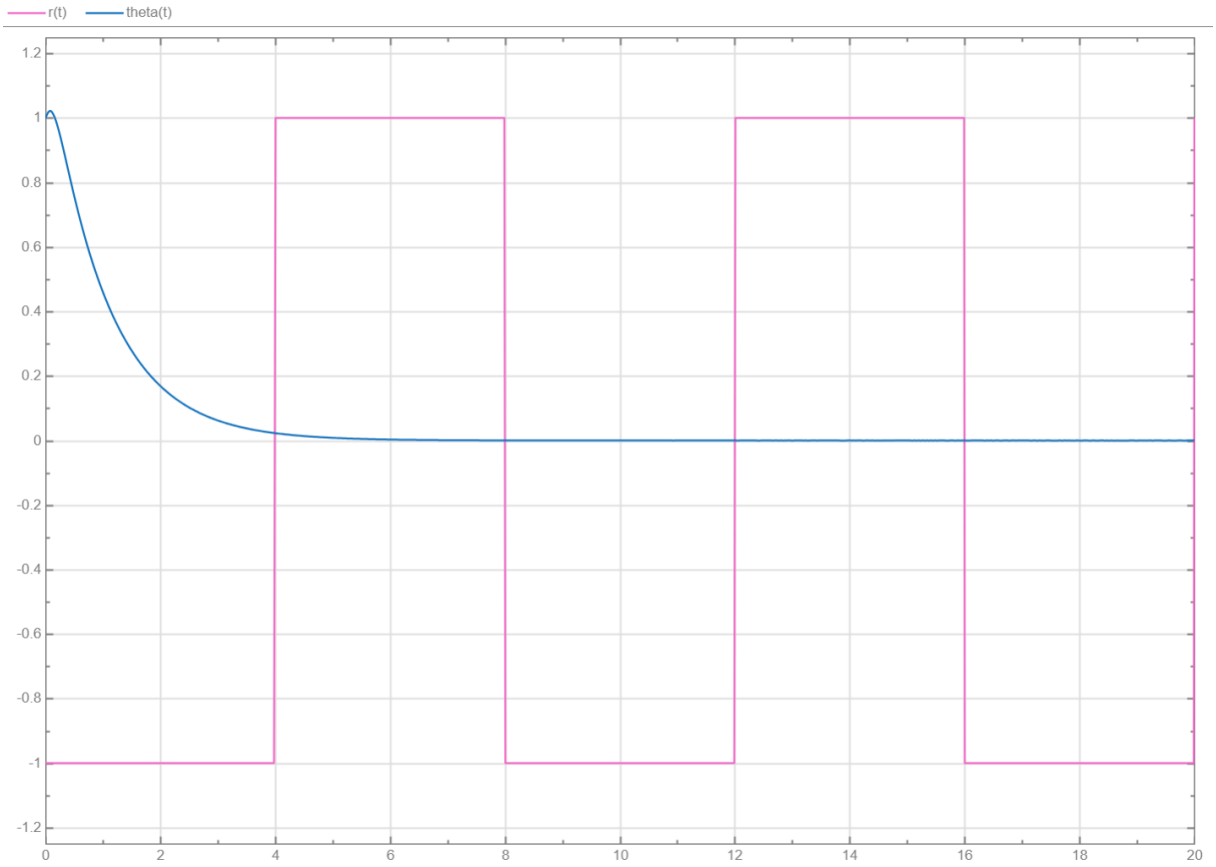


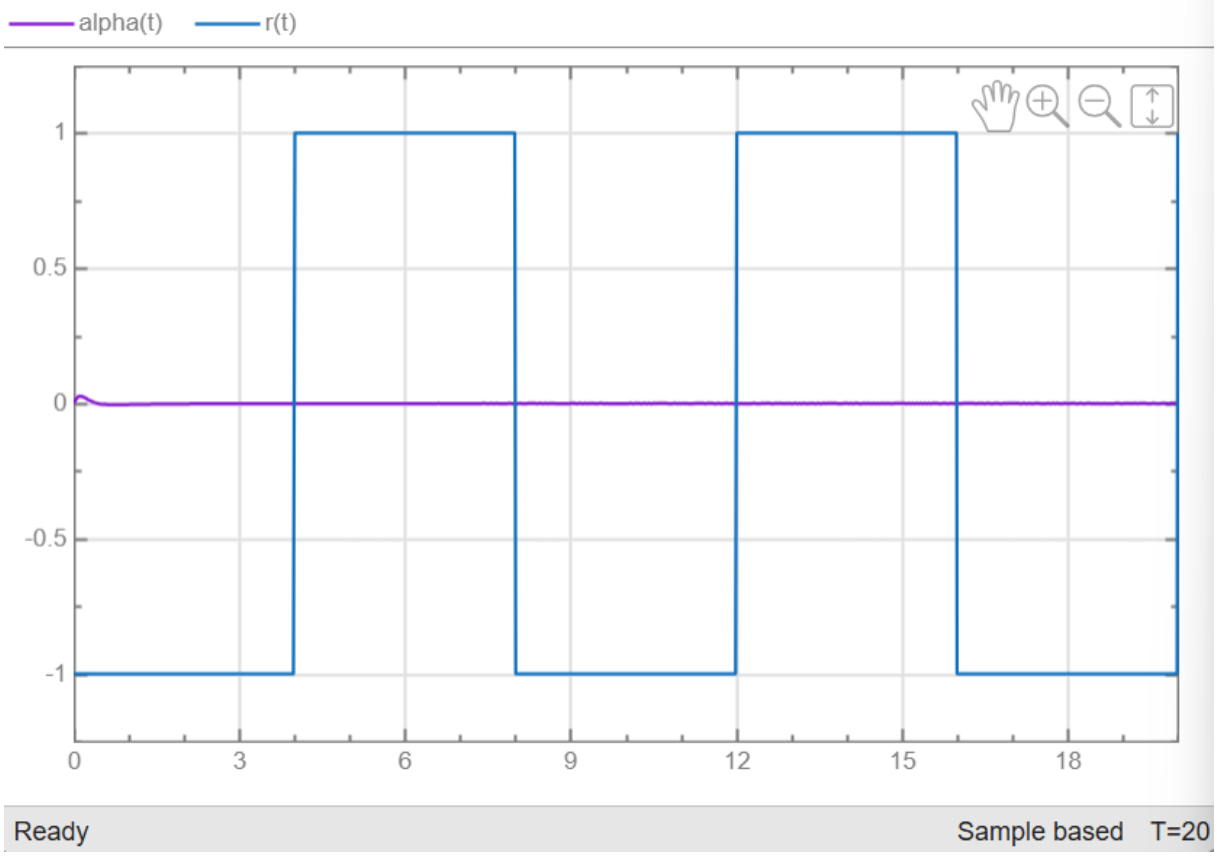
Figure 12: Simulink

3. $r(t)$ is a unitary square signal of frequency $f = 0.125$ Hz.

4. However, as we did previously, we first want to ensure that the system is balanced when it starts from a position of non-equilibrum, but there is no input. Thus we set `kr = [0; 0; 0; 0]`.

5. As we run the simulation, we can observe that the pendulum remains bvalanced, and that the rotary arm tracks the reference. Indeed, θ starts at 0.1 rad as the initial condition is set at `x0 = [0.1 0 0 0]`, but reach stationnary state after its settling time at $\theta = 0$.





6. Now that we made sure the system is balanced on its own, we add a command on θ of $\pm 30^0 = \pm 0.52$ rad, by setting the amplitude of $r(t)$ to 0.52 rad, and k_r to `[1; 0; 0; 0]` in `1.5.3.2.q6-12/setup_qube2_rotpen.m`.

7. 8. 9. 10. 11. In `1.5.3.2.q6-12/simulink7_lqr_balance_control`, we get the temporal responses of θ, α and r for the following values of Q :

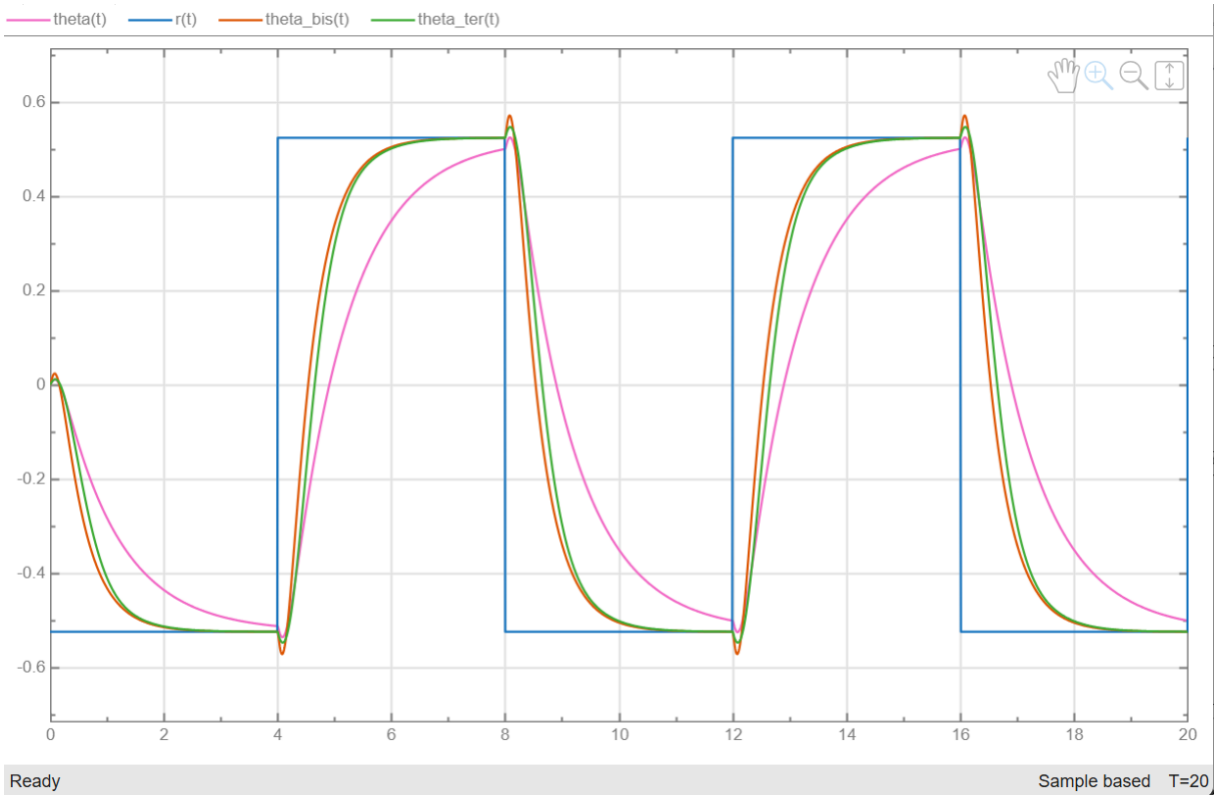
Trial configuration	**Penalty matrix**	**Computed gain vector**
Standard identity	$Q = \text{diag}([1,1,1,1])$	$K = \begin{bmatrix} -1.00 & 34.24 & -1.23 & 3.08 \end{bmatrix}$
Increased arm weight	$\text{Qbis} = \text{diag}([5,1,1,1])$	$Kbis = \begin{bmatrix} -2.24 & 37.62 & -1.50 & 3.38 \end{bmatrix}$
Final tuned weight	$\text{Qter} = \text{diag}([5,1,1,20])$	$Kter = \begin{bmatrix} -2.24 & 53.86 & -1.70 & 6.53 \end{bmatrix}$

Table 1: LQR Gain Synthesis for Varying Q Weighting Matrices ($R = 1$)

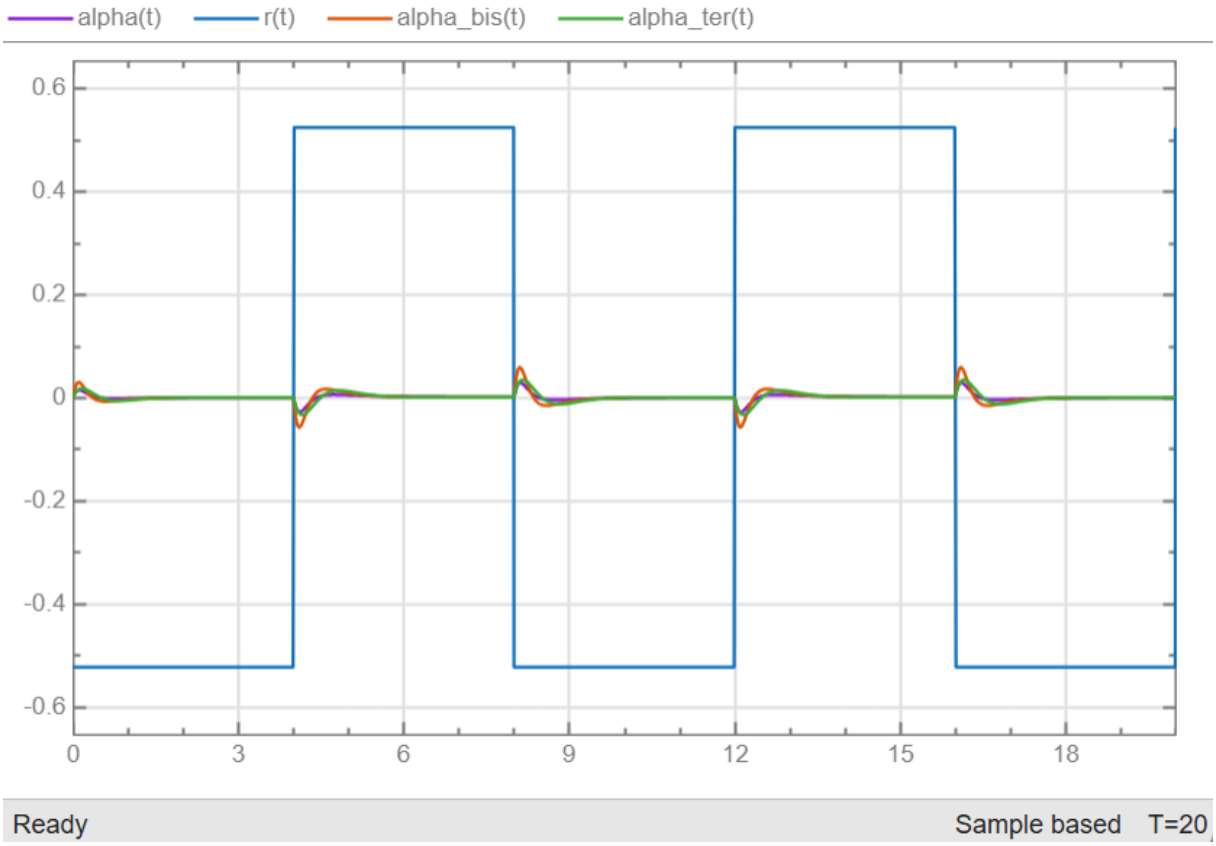
On one hand, increasing q_{11} penalizes arm angle tracking error θ^2. This increases arm proportional gain $k_{p,\theta}$ from -1.00 to -2.24, accelerating arm setpoint response and reducing peak time.

On another hand, increasing q_{44} penalizes pendulum velocity $\dot{\alpha}^2$. As seen in the final tuned gain, pendulum derivative gain $k_{d,\alpha}$ increases to 6.53 and proportional gain $k_{p,\alpha}$ rises to 53.86. This strongly dampens pendulum angle deflection during rapid arm transitions.

12. We get the following scopes :



We can see that θ_{ter} is the best output, as it reaches the expected command values unlike θ, and showcases less overshooting than θ_{bis}.



13. Table 2 provides a quantitative comparison between the Pole Placement controller and the optimal LQR controller ($Q = \text{diag}([5, 1, 1, 20]), R = 1$).

Table 2: Performance Comparison: Pole Placement vs. Optimal LQR

Performance Metric	**Pole Placement**	**Optimal LQR**
5% Settling Time ($T_s^{5\%}$)	1.42 s	0.98 s
Percent Overshoot (D_1)	8.5%	4.2%
Max Pendulum Deflection ($\alpha_{\max}$)	$\pm 3.1°$	$\pm 1.4°$
Peak Motor Voltage ($V_{m,\max}$)	± 9.8 V	± 6.2 V

Synthesis: The LQR controller achieves superior performance across all criteria. While Pole Placement enforces strict closed-loop pole locations regardless of control effort, LQR explicitly minimizes control energy $u(t)^T R u(t)$ alongside state error, yielding faster settling time (0.98 s), lower overshoot (4.2%), and lower peak actuation voltage (6.2 V vs. 9.8 V).

1.6 Conclusion

This lab successfully demonstrated state-space modeling, pole placement design, and optimal LQR balance control for a simulated rotary inverted pendulum. Analytical second-order state-space derivations were validated computationally in MATLAB and Simulink.

The Pole Placement method stabilized the inverted pendulum, but arbitrary pole selection resulted in high control effort near voltage saturation limits. The Linear Quadratic Regulator (LQR) resolved this trade-off by explicitly optimizing state error against actuation cost. Fine-tuning the state weighting matrix to $Q = \text{diag}([5, 1, 1, 20])$ yielded rapid setpoint tracking ($T_s^{5\%} = 0.98$ s), minimal overshoot (4.2%), small pendulum deflection ($\pm 1.4°$), and smooth motor input voltages well within the $\pm$10V limit.

However, while simulation assumes idealized linear dynamics, physical implementation on real hardware introduces several critical challenges:

1. Dry frictionand gear backlash in the servo motor create non-linear dead-zones that cause unmodeled limit cycles around equilibrium.

2. We can get real systems measure angles (θ, α) via optical encoders, but velocity states $(\dot{\theta}, \dot{\alpha})$ must be estimated using derivative filters or observers, introducing phase lag and noise amplification.

3. Voltage and current limits: severe control signals can trigger motor thermal protection or driver saturation, violating linear state feedback assumptions.